



Recipe Manager — Cloud Deployment Case Study

Flask + MongoDB Atlas, containerized with Docker, and deployed to production on AWS ECS Fargate via ECR and Secrets Manager. A workflow walkthrough for interview prep and reference.

Flask

MongoDB Atlas

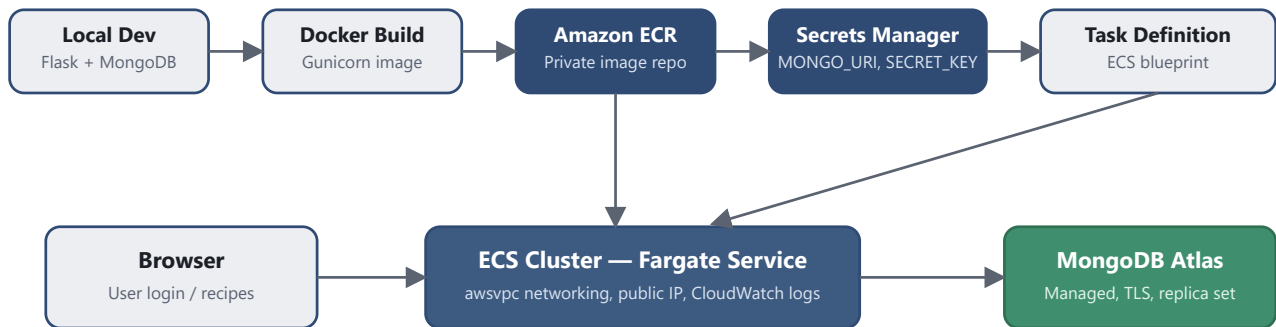
Docker

Amazon ECR

AWS Secrets Manager

ECS Fargate

Architecture Overview



Container image travels left → right through the build pipeline; live request traffic flows bottom row: Browser → Fargate Service → MongoDB Atlas.

Step-by-Step Workflow

1 Local development

Flask app built and tested locally against MongoDB Atlas, with secrets loaded from a local `.env` file via `python-dotenv` — never hardcoded in source.

2 Containerize with Docker

A `Dockerfile` installs dependencies and runs the app with Gunicorn (multiple workers + threads) instead of Flask's dev server. A `.dockerignore` keeps the `virtualenv`, `git` history, and secrets out of the image.

3 Push the image to Amazon ECR

A private ECR repository is created; the local image is tagged with the ECR URI and pushed there, authenticated via a short-lived token from `aws ecr get-login-password`.

4 Store secrets in AWS Secrets Manager

`MONGO_URI` and `SECRET_KEY` are stored as managed secrets, referenced by ARN — never placed in the image or the task definition as plaintext.

5 Register an ECS task definition

Defines the container, CPU/memory (Fargate-compatible sizing), port mapping, the two secrets injected as environment variables at runtime, and a CloudWatch log group for visibility.

6 Create the cluster, security group, and service

An ECS cluster hosts a Fargate service running the task, using `awsvpc` networking with a public IP. A security group opens inbound access to the app's port.

7 Verify and iterate

CloudWatch logs surfaced two real issues fixed along the way: a single-worker Gunicorn process getting stuck on incomplete connections, and a transient MongoDB TLS handshake failure — both diagnosed from log evidence rather than guesswork.

Persistent Storage

Fargate tasks are ephemeral — anything written to the container's local filesystem disappears on restart or redeploy. This app sidesteps that entirely:

- **All application data lives in MongoDB Atlas**, fully decoupled from the container lifecycle. Every task instance reads/writes the same external database.
- **Sessions are stateless** — Flask-Login signs cookies with SECRET_KEY and stores them client-side, so any instance can serve any request with no shared session store needed.
- **Write durability** is enforced via `retryWrites=true&w=majority` in the Mongo connection string — writes are acknowledged by a majority of the replica set before the app returns success.

Lessons Learned / Troubleshooting

Quoted .env values break Docker's --env-file. python-dotenv strips quotes automatically; Docker does not. A quoted `MONGO_URI` passes a literal quote character into the connection string, which PyMongo rejects outright.

A single Gunicorn sync worker is fragile in production. With no worker/thread tuning, one slow or incomplete connection (e.g. a port scanner opening a socket and going silent) blocks the entire app until the 30s default timeout kills the worker. Fixed with `--workers 2 --threads 2 --timeout 60`.

Exposing a raw container port to 0.0.0.0/0 invites scanner traffic. Without a load balancer in front, bots constantly probe open ports. The durable fix is an Application Load Balancer with the security group locked down to ALB-only traffic, plus a stable DNS name instead of a Fargate task's ephemeral public IP.

Browser-specific HTTPS-upgrade behavior can look like an infra bug. Chrome's "Always use secure connections" mode tried to upgrade a plain-HTTP address to HTTPS, hanging until timeout — while Firefox and Edge worked immediately. Confirmed via a raw TCP test (Test-NetConnection) that the deployment itself was healthy.

A TLS alert from MongoDB is not automatically an AWS permissions problem. If the client received a TLS alert *from Atlas*, the network path (security group, routing, DNS) already worked — the rejection happened at the TLS/Atlas layer. Correctly identifying which layer failed avoided wasted time second-guessing IAM policies that were already correct.